



PRÉ-RENTRÉE 2026-2027

Séance 1 - Cahier élève

Variables, types, booléens et conditions

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ comprendre l'ordre d'évaluation d'une affectation ;
- ☐ distinguer affectation et comparaison ;
- ☐ identifier les types de base ;
- ☐ écrire une condition et sa négation ;
- ☐ programmer un classificateur robuste ;

Rituel sans ordinateur

Question 1

Tracer sans exécuter :

```
a = 4
b = a + 2
a = 10
```

Que valent `a` et `b` à la fin ?

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Écrire en Python la négation exacte de :

```
age >= 18
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

État et affectation

Une variable est un nom associé à une valeur dans l'état courant du programme. Pour `x = expression`, Python :

1. évalue l'expression à droite ;
2. stocke la valeur obtenue sous le nom à gauche.

Ainsi, après `a = 3` ; `b = a` ; `a = 5`, `b` vaut encore `3`.

Types de base

Type	Exemple	Usage
<code>int</code>	<code>12</code> , <code>-4</code>	entier
<code>float</code>	<code>3.5</code>	approximation d'un réel
<code>str</code>	<code>"NSI"</code>	texte
<code>bool</code>	<code>True</code> , <code>False</code>	condition
<code>NoneType</code>	<code>None</code>	absence de valeur

Booléens et conditions

- `and` exige que les deux conditions soient vraies ;
- `or` exige qu'au moins une condition soit vraie ;
- `not` nie la condition ;
- la négation de `x > 5` est `x <= 5`.

```
if temperature < 0:
    etat = "gel"
elif temperature <= 30:
    etat = "normal"
else:
    etat = "alerte"
```

Activité commune - tables de trace

A. Affectations

Compléter les états successifs.

```
a = 3
b = a
a = 5
c = a + b
```

Ligne exécutée			
départ	non défini	non défini	non défini
a = 3			
b = a			
a = 5			
c = a + b			

B. Conditions aux bornes

Pour chacune des valeurs -1, 0, 5, 10, 11, indiquer la valeur de :

$(x > 0) \text{ and } (x < 10)$

x	résultat	justification
-1		
0		
5		
10		
11		

Parcours Fondations - Ahmad

1. Corriger le programme suivant :

```
age = "16"  
print(age + 1)
```

2. Compléter pour afficher `pair`, `impair` ou `nul`.

```
n = -3  
# À compléter
```

3. Écrire la négation Python de chacune des conditions :

- `x >= 0` ;
- `age < 18` ;
- `(x > 0) and (x < 10)`.

4. Écrire un programme qui classe une température en `gel`, `normal` ou `alerte`.

Parcours Fiabilisation - Ahmed

1. Écrire une condition vraie exactement lorsque `x` n'appartient pas à l'intervalle `[0, 10]`.
2. Simplifier la négation de `(x >= 2) and (x <= 8)`.
3. Corriger un programme comportant une branche inaccessible.
4. Écrire des tests sur les valeurs limites `-1`, `0`, `30`, `31` du classificateur de température.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :